

Lecture 9

บท: Induction (4): Strong Induction and Recursion Correctness Proof

วิชา: 819605 Discrete Mathematics

อาจารย์ผู้สอน: ปพนธีร์ แยมโซติ | วิศวกรรมคอมพิวเตอร์ | ภาคปลาย ปีการศึกษา 2568

เป้าหมายการเรียนรู้ Week 7

1. พิสูจน์ strong induction ได้
2. สามารถพิสูจน์ความถูกต้องของ algorithm ที่เป็น recursion ได้ (ผ่านตัวอย่าง Divide-and-Conquer)

1 การพิสูจน์แบบ Strong Induction

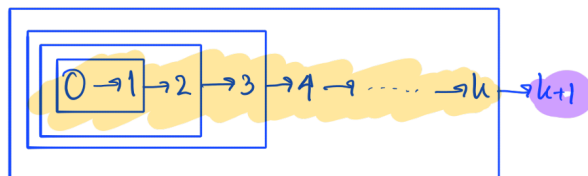
ในบางครั้ง การอยากได้คำตอบของขั้นที่ $k + 1$ อาจจะได้เกิดจากผลขั้นที่ k อย่างเดียว แต่อาจจะ

- เกิดจากหลาย ๆ ขั้นต่ำกว่ามาช่วยกัน หรือ
- เกิดจากขั้นเดียวแหละ แต่ระบุแน่นอนไม่ได้ว่าขั้นที่เท่าไร เช่นผลของขั้นที่ $n = 100$ อาจเกิดมาจากขั้นที่ $n = 25$ แต่ผลจากขั้นที่ $n = 150$ อาจเกิดมาจากผลของขั้นที่ $n = 149$

ทำให้เราไม่สามารถสมมติแค่ขั้น k เป็นสมมติฐานขั้นอุปนัยได้ กล่าวคือ เราไม่สามารถอุปนัยด้วยโครงสร้างเดิมด้านล่างได้

$$0 \longrightarrow 1 \longrightarrow 2 \longrightarrow \dots \longrightarrow k \longrightarrow k + 1 \longrightarrow \dots$$

แต่เราสามารถใช้โครงสร้างด้านล่างนี้ทดแทนได้



โครงสร้างการเขียนพิสูจน์ Strong Induction

สิ่งที่อยากพิสูจน์: เรากำลังจะพิสูจน์ว่าข้อความ $P(n)$ เป็นจริงสำหรับทุก ๆ $n \in \mathbb{N}$ ด้วยอุปนัย(แบบแข็ง)บน n

ขั้นฐาน: พิสูจน์ว่า $P(0)$ เป็นจริง (หรืออาจจะฐานอื่นขึ้นอยู่กับโจทย์)

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า $P(0)$ จนถึง $P(k)$ เป็นจริง

...(พิสูจน์ว่า $P(k + 1)$ เป็นจริง)...

ดังนั้น $P(k + 1)$ เป็นจริง

Example 1. ถ้ากำหนดให้ $a_n = 5a_{n-1} - 6a_{n-2}$ โดยที่กำหนดให้ $a_0 = 1$ และ $a_1 = 2$ จงพิสูจน์ว่า $a_n = 2^n$ สำหรับทุก ๆ จำนวนนับ n

Example 2. ถ้าเรามีแผ่นช็อคโกแลตบาร์ขนาด $n \times 1$ จงพิสูจน์ว่าเราจะต้องใช้การหักแบ่ง $n - 1$ ครั้งเพื่อให้เหลือขนาด 1×1 ทุกชิ้น

Example 3. ในเกมหนึ่ง ร้านค้าให้ซื้อเหรียญได้แค่ 2 แบบ คือแพ็คเกจ 4 เหรียญ และแพ็คเกจ 5 เหรียญ จงแสดงว่า ไม่ว่าผู้เล่นจะอยากได้เหรียญจำนวนเท่าใดก็ตาม ถ้าเป็นจำนวนตั้งแต่ 12 เหรียญขึ้นไป จะสามารถซื้อให้ได้พอดีเสมอ

2 Divide-and-Conquer: Merge Sort

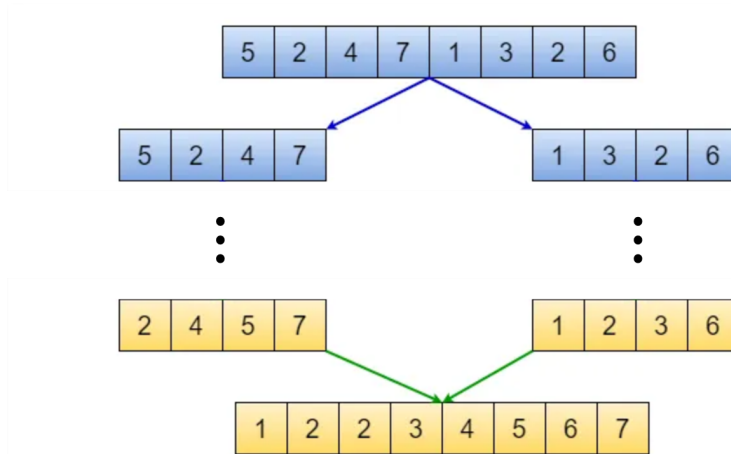
แนวคิดหลัก: Divide-and-Conquer (D&C)

Divide-and-Conquer คือแพตเทิร์นการออกแบบอัลกอริทึม/พิสูจน์ความถูกต้องที่มี 3 ขั้นตอน:

1. **Divide**: แบ่งปัญหามาขนาด n ออกเป็นปัญหาย่อยที่ *เล็กกว่าเดิม* (เช่น ประมาณครึ่งหนึ่ง)
2. **Conquer**: แก้ปัญหาย่อยด้วย recursion (เรียกตัวเอง)
3. **Combine**: รวมคำตอบย่อยให้เป็นคำตอบของปัญหาเดิม

หัวใจของการพิสูจน์ **correctness**: ระบุ *สเปก* ให้ชัด และพิสูจน์ว่า **Combine** ไม่ทำลายความถูกต้อง เมื่อสมมติว่า **Conquer** ทำถูกในปัญหาที่เล็กกว่า

2.1 Merge Sort



เราจะมองลิสต์เป็นลำดับ $A = (a_1, a_2, \dots, a_n)$ และใช้ $|A|$ แทนความยาวของลิสต์

สเปกที่ต้องพิสูจน์

Spec ของ MergeSort: สำหรับทุกลิสต์ A , ผลลัพธ์ $\text{MergeSort}(A)$ ต้องมีคุณสมบัติ 2 ข้อพร้อมกัน

1. $\text{MergeSort}(A)$ เป็นลิสต์ที่ **sorted**
2. $|\text{MergeSort}(A)| = |A|$ (คือเป็นการ “เรียงใหม่” โดยไม่ทำสมาชิกหาย/เพิ่ม)

2.2 อัลกอริทึม (Pseudocode)

Pseudocode: MergeSort และ Merge

MergeSort(A):

if $|A| \leq 1$ **then return** A

 แบ่ง A เป็น L, R โดย $|L| = \lfloor |A|/2 \rfloor$ และ $|R| = \lceil |A|/2 \rceil$

return Merge(MergeSort(L), MergeSort(R))

Merge(L, R): (สมมติว่า L และ R sorted แล้ว)

$i \leftarrow 1, j \leftarrow 1, \text{out} \leftarrow []$

while $i \leq |L|$ **and** $j \leq |R|$ **do**

if $L[i] \leq R[j]$ **then** append $L[i]; i \leftarrow i + 1$

else append $R[j]; j \leftarrow j + 1$

 append ส่วนที่เหลือของรายการที่ยังไม่หมด

return out

2.3 พิสูจน์ความถูกต้องของ Merge

Claim 4 (Correctness of Merge). ถ้า L และ R เป็นลิสต์ที่ sorted แล้ว ให้ $M = \text{Merge}(L, R)$. แล้ว

1. M เป็น sorted
2. $|M| = |L| + |R|$

Proof idea. พิสูจน์ด้วย strong induction บน $s = |L| + |R|$. ใกล้เคียงคือดู “สมาชิกตัวแรก” ที่ Merge ปล่อยออกมา (ต้องเป็นค่าที่เล็กสุดของหัวทั้งสองลิสต์) แล้วลดขนาดปัญหาเหลือ $s - 1$.

2.4 พิสูจน์ความถูกต้องของ MergeSort

Theorem 5 (Correctness of MergeSort). สำหรับทุกลิสต์ A , ลิสต์ $MergeSort(A)$ เป็นลิสต์ตามที่ต้องการ (เรียงจากน้อยไปมาก และสมาชิกอยู่ครบ)

Proof idea. ทำ induction บน $n = |A|$ (เหมาะกับ recursion เพราะทุกครั้งที่เรียกตัวเอง ขนาดปัญหาจะลดลงจริง) โดย induction step ต้อง “ปิดงาน” ด้วย Lemma/Claim ของ Merge ที่พิสูจน์ไว้แล้ว

3 การบ้าน Week 9

การบ้านให้เวลาทำ 2 สัปดาห์ แต่แนะนำให้พยายามทำเรื่อย ๆ อย่าต้องไว้ทำคืนสุดท้าย เพราะไม่ทันแน่ ๆ

- กำหนดให้ R เป็นเซตที่นิยามแบบเวียนเกิดดังนี้

- $1 \in R$
- สำหรับ $n \in \mathbb{N}$ ใด ๆ ถ้า $n \in R$ แล้ว $5n, n^2 \in R$

จงพิสูจน์ว่า ทุก ๆ สมาชิก $x \in R$ จะมี $q \in \mathbb{N}$ ที่ทำให้ $x = 4q + 1$
(กล่าวคือทุกสมาชิกใน R คือพวกจำนวนนับที่หารด้วย 4 แล้วเหลือเศษ 1)

- นิยามฟังก์ชันการ reverse สายอักขระ (string) ดังนี้

- $\text{reverse}(\epsilon) = \epsilon$ โดยที่ ϵ คือตัวสายอักขระว่าง (ความยาว 0)
- สำหรับตัวอักขระ a และสายอักขระ w จะได้ว่า $\text{reverse}(wa) = a \cdot \text{reverse}(w)$ โดย $\cdot$ หมายถึงการต่อข้อความ (concatenation)

ตัวอย่างเช่น $\text{reverse}(abcd) = dcba$ หรือ $\text{reverse}(a) = a$ หรือถ้ามองในระดับการเวียนเกิด จะได้ว่า

$$\text{reverse}(abcd) = d \cdot \text{reverse}(abc) = dc \cdot \text{reverse}(ab) = dcb \cdot \text{reverse}(a) = dcba$$

จงพิสูจน์ว่า $\text{reverse}(xy) = \text{reverse}(y) \cdot \text{reverse}(x)$ สำหรับสายอักขระ x และ y ใด ๆ

- จงพิสูจน์ว่าอัลกอริทึม FindFirst ที่นิยามแบบ recursion ในเฉลยการบ้านของ week 5 (Reading Assignment Week 6) ทำงานได้ถูกต้อง
- จงพิสูจน์ว่าอัลกอริทึม FindLast ที่นิยามแบบ recursion ในเฉลยการบ้านของ week 5 (Reading Assignment Week 6) ทำงานได้ถูกต้อง
- จงพิสูจน์ว่านิยามแบบ recursion ของเซตของ bit strings ที่มี 0 ติดกันไม่เกิน 2 ตัวที่ให้ในเฉลยการบ้าน week 5 (Reading Assignment Week 6) ถูกต้อง ซึ่งนิยามที่ให้ไว้เฉลยคือ

- $0, 1, 00, 01, 10, 11, 001, 010, 011, 100, 101, 110, 111 \in R$
- สำหรับ bit string x ใด ๆ
 - ถ้า $x \in R$ และ x ลงท้ายด้วย 1 แล้ว $x00, x11, x10, x11 \in R$
 - ถ้า $x \in R$ และ x ลงท้ายด้วย 00 แล้ว $x10, x11 \in R$
 - ถ้า $x \in R$ และ x ลงท้ายด้วย 10 แล้ว $x01, x10, x11 \in R$

(คำเตือน: อย่าลืมว่าต้องพิสูจน์ 2 ขา)

- ในโลกคอมพิวเตอร์ เราต้องใช้เลขฐานสองเป็นตัวแทนของจำนวนนับเพื่อให้คอมพิวเตอร์คำนวณได้ เช่น $6 + 10 = 110_2 + 1010_2 = 10000_2 = 16$ และนิยามของเลขฐานสองคือ

$$(x_n x_{n-1} \dots x_2 x_1 x_0)_2 = x_n \times 2^n + x_{n-1} \times 2^{n-1} + \dots + x_2 \times 2^2 + x_1 \times 2^1 + x_0 \times 2^0$$

เช่น $110_2 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 2^2 + 2^1$ หรือ $1010_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 2^3 + 2^1$

ซึ่งคำถามที่ควรถามเป็นอันดับแรกก็คือ "เราสามารถแสดงผลจำนวนนับเป็นเลขฐานสองได้เสมอหรือไม่" (เพราะไม่เช่นนั้น จะกลายเป็นว่าคอมพิวเตอร์จะไม่สามารถคำนวณบางจำนวนไม่ได้) ให้นักศึกษาตั้งข้อคำถามดังกล่าวเป็นข้อความทางคณิตศาสตร์ที่จะต้องพิสูจน์ พร้อมทั้งพิสูจน์ข้อความดังกล่าว